

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-a3870dca2a8bf4dc3.jsonl`  
- SHA-256: `8c63b1db03f5fa0720e4da8f028f1ecbd0cf4f7ed239876af2eb5646cb42417c`  
- Source modified: `2026-06-10T16:28:12+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:25:50.945Z)

You are an adversarial verifier. A code reviewer audited the repo at
C:/Users/avidu/Projects/gmail-attachments-to-gdrive and reported this finding:

TITLE: GmailApp.getMessageById throws on deleted messages, wedging incremental sync until the cursor expires
SEVERITY CLAIMED: high
FILE: src/gmailHistory.ts (line 70-77)
DESCRIPTION: threadsForMessageIds assumes a falsy return for a message that no longer exists ('Messages that can no longer be fetched ... are skipped'). In the real GAS runtime GmailApp.getMessageById throws an exception for an id that is invalid or permanently deleted; it does not return null (the unit tests stub it to return null, which is why this passes). A message recorded in history and then deleted/purged before the run (user deletes junk right after receipt ? common at a 20-minute trigger cadence) makes every incremental run throw. The exception propagates to the catch in runSyncOnce BEFORE state.setHistoryId is called, so the cursor never advances and every subsequent run re-fetches the same dead id and throws again. The wedge persists until the stored historyId itself expires (typically a week or more), during which no new attachments sync; if retention exceeds 30 days, the eventual 30-day rewind also leaves a permanent gap.

```
EVIDENCE QUOTED: for (const id of messageIds) {  
  const message = GmailApp.getMessageById(id);  
  if (!message) {  
    continue;  
  }  
}
```

SUGGESTED FIX: Wrap the per-id fetch in try/catch and skip ids that throw (log at debug level). Add a test whose fake getMessageById throws, mirroring the real runtime contract.

Your job is to try to REFUTE it. Your lens: existing-mitigation: check whether something ELSE in the repo (a test, a guard, a caller-side check, a hook, config) already prevents or detects the claimed problem.

Read the actual code in the repo (and tests/config where relevant). If after reading you cannot confirm the problem is real, set isReal=false. If it is real but over/under-stated, set adjustedSeverity accordingly. Be strict: plausible-sounding but unconfirmed = refuted.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
 - If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
 - Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
 - Do not use WebSearch/WebFetch and do not send any repo content to any external service.
- Structured output only.

2. user (2026-06-10T16:26:00.788Z)

```
LICENSE
PRIVACY.md
README.md
ROADMAP.md
appsscript.json
docs
esbuild.config.mjs
eslint.config.mjs
html
jest.config.js
package-lock.json
package.json
src
tests
tsconfig.json
config.ts
contentTypes.ts
drive.ts
entry.ts
gmailHistory.ts
historySync.ts
logger.ts
naming.ts
notify.ts
processed.ts
query.ts
quota.ts
state.ts
stats.ts
sync.ts
types
ui.ts
```

3. user (2026-06-10T16:26:01.454Z)

```
1      import {
2          collectAddedMessageIds,
3          isHistoryExpired,
4          nextHistoryId,
5          type HistoryListResponse,
6      } from "../historySync";
7      import { logDebug } from "../logger";
8
9      type GmailThread = GoogleAppsScript.Gmail.GmailThread;
10
11      /** A bounded number of history pages to walk in a single run. */
12      const MAX_HISTORY_PAGES = 20;
13
14      /** The mailbox's current historyId, used to seed incremental sync. */
15      export function getCurrentHistoryId(): string | null {
16          const profile = Gmail.Users.getProfile("me");
17          return profile.historyId ?? null;
18      }
19
20      /** What an incremental scan yields: new message ids and the id to store next. */
21      export interface IncrementalChanges {
22          messageIds: string[];
23          historyId: string;
24      }
25
26      /**
27       * Lists messages added since `startHistoryId` via the Gmail advanced service,
28       * paging through results. Returns null when the history has expired (the caller
```

```

29     * must fall back to a full sync). Only `messageAdded` events are requested.
30     */
31     export function listAddedSince(
32         startHistoryId: string,
33     ): IncrementalChanges | null {
34         const responses: HistoryListResponse[] = [];
35         let pageToken: string | undefined;
36         try {
37             for (let page = 0; page < MAX_HISTORY_PAGES; page++) {
38                 const response = Gmail.Users.History.list("me", {
39                     startHistoryId,
40                     historyTypes: ["messageAdded"],
41                     pageToken,
42                 }) as HistoryListResponse;
43                 responses.push(response);
44                 if (!response.nextPageToken) {
45                     break;
46                 }
47                 pageToken = response.nextPageToken;
48             }
49         } catch (e) {
50             if (isHistoryExpired(e)) {
51                 logDebug("History id expired; full sync required.");
52                 return null;
53             }
54             throw e;
55         }
56         return {
57             messageIds: collectAddedMessageIds(responses),
58             historyId: nextHistoryId(responses, startHistoryId),
59         };
60     }
61
62     /**
63     * Resolves the distinct Gmail threads containing the given message ids. Threads
64     * are de-duplicated by id so a multi-message thread is processed once. Messages
65     * that can no longer be fetched (deleted since the change was recorded) are
66     * skipped.
67     */
68     export function threadsForMessageIds(messageIds: string[]): GmailThread[] {
69         const threads = new Map<string, GmailThread>();
70         for (const id of messageIds) {
71             const message = GmailApp.getMessageById(id);
72             if (!message) {
73                 continue;
74             }
75             const thread = message.getThread();
76             threads.set(thread.getId(), thread);
77         }
78         return Array.from(threads.values());
79     }
80

```

4. user (2026-06-10T16:26:12.309Z)

```

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts:31:export function
listAddedSince(
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\gmailHistory.ts:68:export function
threadsForMessageIds(messageIds: string[]): GmailThread[] {
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\state.ts:101:export function
setHistoryId(id: string): void {
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts:14: listAddedSince,
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts:15: threadsForMessageIds,
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts:166: const changes =
listAddedSince(historyId);

```

```

C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts:176:      const threads =
threadsForMessageIds(changes.messageIds);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts:189:
state.setHistoryId(changes.historyId);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\src\sync.ts:222:
state.setHistoryId(candidateHistoryId);
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts:3:
listAddedSince,
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts:4:
threadsForMessageIds,
C:\Users\avidu\Projects\gmail-attachments-to-
gdrive\tests\gmailHistory.test.ts:25:describe("listAddedSince", () => {
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts:42:
expect(listAddedSince("50")).toEqual({
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts:54:
expect(listAddedSince("1")).toBeNull();
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts:62:      expect(()
=> listAddedSince("1")).toThrow("500");
C:\Users\avidu\Projects\gmail-attachments-to-
gdrive\tests\gmailHistory.test.ts:66:describe("threadsForMessageIds", () => {
C:\Users\avidu\Projects\gmail-attachments-to-gdrive\tests\gmailHistory.test.ts:76:      const
threads = threadsForMessageIds(["a", "b", "c", "gone"]);

```

5. user (2026-06-10T16:26:12.821Z)

```

1      import {
2          ENABLE_INCREMENTAL_SYNC,
3          MAX_FILES_ADD_PER_DAY,
4          MAX_RUNTIME_MS,
5          MAX_SYNC_TRIGGERS,
6          ONE_DAY,
7          ONE_MINUTE,
8          SIX_HOURS,
9          THIRTY_DAYS,
10     } from "./config";
11     import { updateDrive, type UpdateResult } from "./drive";
12     import {
13         getCurrentHistoryId,
14         listAddedSince,
15         threadsForMessageIds,
16     } from "./gmailHistory";
17     import { logDebug, logError } from "./logger";
18     import { maybeSendDailySummary } from "./notify";
19     import { addProcessedId } from "./processed";
20     import { buildSearchQuery } from "./query";
21     import { hasTime, isSameCalendarDay } from "./quota";
22     import { dayKey, recordRun } from "./stats";
23     import * as state from "./state";
24
25     const SYNC_HANDLER = "syncUserDrives";
26
27     /** Schedules a one-shot run of syncUserDrives `afterMs` from now. */
28     export function newSyncTrigger(afterMs: number): void {
29         ScriptApp.newTrigger(SYNC_HANDLER).timeBased().after(afterMs).create();
30     }
31
32     /** All currently-installed triggers for the sync handler. */
33     function getSyncTriggers(): GoogleAppsScript.Script.Trigger[] {
34         return ScriptApp.getProjectTriggers().filter(
35             (t) => t.getHandlerFunction() === SYNC_HANDLER,
36         );
37     }
38
39     /** Removes every project trigger (used when the user re-runs setup). */
40     export function deleteAllTriggers(): void {

```

```

41     for (const trigger of ScriptApp.getProjectTriggers()) {
42         ScriptApp.deleteTrigger(trigger);
43     }
44 }
45
46 function deleteTriggers(triggers: GoogleAppsScript.Script.Trigger[]): void {
47     for (const trigger of triggers) {
48         ScriptApp.deleteTrigger(trigger);
49     }
50 }
51
52 /** Defense-in-depth: never let sync triggers exceed the cap (limit is 20). */
53 function pruneSyncTriggers(max: number): void {
54     const triggers = getSyncTriggers();
55     for (let i = max; i < triggers.length; i++) {
56         ScriptApp.deleteTrigger(triggers[i]);
57     }
58 }
59
60 /**
61  * The background worker. A per-user lock ensures only one run executes at a time
62  * for THIS user, so concurrent triggers cannot stack file copies or trigger
63  * generations. It is deliberately a user lock, not a script lock: the web app
64  * runs as the accessing user (executeAs USER_ACCESSING), so a script-wide lock
65  * would serialise every user's sync into one global queue.
66  */
67 export function syncUserDrives(): void {
68     const lock = LockService.getUserLock();
69     if (!lock.tryLock(0)) {
70         logDebug("Another sync run is in progress; skipping.");
71         return;
72     }
73     try {
74         runSyncOnce();
75     } finally {
76         // Bound the trigger count regardless of which path we took, then release.
77         pruneSyncTriggers(MAX_SYNC_TRIGGERS);
78         lock.releaseLock();
79     }
80 }
81
82 /**
83  * A single sync pass:
84  * 1. reschedules a fan-out of backup triggers (self-healing if one is lost),
85  * 2. syncs forward in 30-day windows until it runs out of time, hits the
86  *    daily file cap, or fully catches up,
87  * 3. persists progress and schedules the next run.
88  */
89 function runSyncOnce(): void {
90     const today = new Date();
91
92     // On the first run of a new calendar day, summarise the previous day.
93     maybeSendDailySummary(today);
94
95     // Recover state if the debug flag was cleared (e.g. properties were wiped).
96     const debug = state.getDebug();
97     if (debug == null || debug === "0") {
98         state.setDebug("1");
99         state.setLastSynced(new Date(today.getTime() - 365 * ONE_DAY).toJSON());
100     }
101
102     // Create the backup triggers BEFORE deleting the current ones, so a failure
103     // here can never leave us with no scheduled run.
104     const previousTriggers = getSyncTriggers();
105     try {

```

```

106     newSyncTrigger(60 * ONE_MINUTE);
107     newSyncTrigger(SIX_HOURS);
108     newSyncTrigger(2 * SIX_HOURS);
109     newSyncTrigger(4 * SIX_HOURS);
110     newSyncTrigger(8 * SIX_HOURS);
111     newSyncTrigger(28 * SIX_HOURS);
112 } catch (e) {
113     // Could not install new triggers; leave the old ones and try again later.
114     logError("Could not install triggers: " + e);
115     return;
116 }
117 deleteTriggers(previousTriggers);
118
119 const filesAddedToday = state.GetFilesAddedToday();
120 let added = filesAddedToday.added;
121 const lastCountDate = new Date(Date.parse(filesAddedToday.date));
122
123 if (isSameCalendarDay(lastCountDate, today) && added >= MAX_FILES_ADD_PER_DAY) {
124     logDebug("Used up all quota for today. Can't add more files.");
125     newSyncTrigger(60 * ONE_MINUTE);
126     return;
127 }
128 if (!isSameCalendarDay(lastCountDate, today)) {
129     added = 0;
130 }
131
132 const structure = state.getFolderStructure();
133 let lastSynced = state.getLastSynced();
134
135 // Load the dedup set once; thread it through the run and persist at the end.
136 let processedIds = state.getProcessedIds();
137 const processedSet = new Set(processedIds);
138
139 // This run's contribution to the day's stats.
140 const addedAtStart = added;
141 let runErrors = 0;
142 let runLastError = "";
143
144 // Folds one updateDrive result into this run's accumulators.
145 const applyResult = (result: UpdateResult): void => {
146     added += result.addedFiles;
147     runErrors += result.errors;
148     if (result.lastError) {
149         runLastError = result.lastError;
150     }
151     for (const id of result.processedIds) {
152         if (!processedSet.has(id)) {
153             processedSet.add(id);
154             processedIds = addProcessedId(processedIds, id);
155         }
156     }
157 };
158
159 try {
160     // Incremental mode is opt-in (ENABLE_INCREMENTAL_SYNC) so it rolls out
161     // gradually over the proven date-window backfill. A stored historyId only
162     // exists once a backfill has caught up while the flag was on.
163     const historyId = ENABLE_INCREMENTAL_SYNC ? state.getHistoryId() : null;
164     if (historyId) {
165         // Incremental: process only messages added since the stored historyId.
166         const changes = listAddedSince(historyId);
167         if (changes == null) {
168             // History expired: clear the cursor and re-scan a bounded recent window
169             // (the next catch-up re-captures a fresh historyId). Overlap is safe ?
170             // dedup by message id and the Drive filename guard prevent re-copying.

```

```

171     state.clearHistoryId();
172     lastSynced = new Date(today.getTime() - THIRTY_DAYS).toJSON();
173     logDebug("History expired; reverting to date-window sync.");
174 } else {
175     if (changes.messageIds.length > 0 && added < MAX_FILES_ADD_PER_DAY) {
176         const threads = threadsForMessageIds(changes.messageIds);
177         if (threads.length > 0) {
178             applyResult(
179                 updateDrive(
180                     threads,
181                     structure,
182                     MAX_FILES_ADD_PER_DAY - added,
183                     processedSet,
184                 ),
185             );
186         }
187     }
188     // Advance the cursor so the next run only sees newer changes.
189     state.setHistoryId(changes.historyId);
190 }
191 } else {
192     // Backfill: walk forward in 30-day windows until caught up to today.
193     // Capture the mailbox historyId BEFORE processing so mail arriving
194     // mid-run is still picked up by the first incremental scan.
195     const candidateHistoryId = ENABLE_INCREMENTAL_SYNC
196         ? getCurrentHistoryId()
197         : null;
198     let fullSynced = false;
199     while (
200         hasTime(new Date(), today, MAX_RUNTIME_MS) &&
201         added < MAX_FILES_ADD_PER_DAY &&
202         !fullSynced
203     ) {
204         const maxToAdd = MAX_FILES_ADD_PER_DAY - added;
205         const search = buildSearchQuery(lastSynced, new Date());
206         logDebug("Query: " + search.query + " files asking: " + maxToAdd);
207
208         const threads = GmailApp.search(search.query);
209         if (threads.length > 0) {
210             applyResult(updateDrive(threads, structure, maxToAdd, processedSet));
211         }
212
213         fullSynced = search.isToday;
214         if (added < MAX_FILES_ADD_PER_DAY) {
215             lastSynced = fullSynced
216                 ? today.toJSON()
217                 : new Date(Date.parse(lastSynced) + THIRTY_DAYS).toJSON();
218         }
219     }
220     // Caught up: switch future runs to the cheaper incremental path.
221     if (candidateHistoryId && fullSynced && added < MAX_FILES_ADD_PER_DAY) {
222         state.setHistoryId(candidateHistoryId);
223         logDebug("Backfill complete; stored historyId " + candidateHistoryId);
224     }
225 }
226 } catch (e) {
227     logError("Some error in execution: " + e);
228     state.setLastError(String(e));
229     runErrors++;
230     runLastError = String(e);
231 }
232
233 // Fold this run into the day's accumulator.
234 const dayStats = recordRun(state.getRunStats(dayKey(today)), {
235     filesAdded: added - addedAtStart,

```

```

236         errors: runErrors,
237         lastError: runLastError,
238     });
239     state.setRunStats(dayStats);
240
241     state.setProcessedIds(processedIds);
242     state.setLastSynced(lastSynced);
243     state.setFilesAddedToday({ added, date: today.toJSON() });
244     newSyncTrigger(20 * ONE_MINUTE);
245 }
246

```

6. user (2026-06-10T16:26:34.485Z)

```

1     import {
2         getCurrentHistoryId,
3         listAddedSince,
4         threadsForMessageIds,
5     } from "../src/gmailHistory";
6
7     const g = globalThis as unknown as Record<string, unknown>;
8
9     describe("getCurrentHistoryId", () => {
10         it("returns the profile's historyId", () => {
11             g.Gmail = {
12                 Users: { getProfile: () => ({ historyId: "99" }), History: { list: jest.fn() } },
13             };
14             expect(getCurrentHistoryId()).toBe("99");
15         });
16
17         it("returns null when the profile has no historyId", () => {
18             g.Gmail = {
19                 Users: { getProfile: () => ({}), History: { list: jest.fn() } },
20             };
21             expect(getCurrentHistoryId()).toBeNull();
22         });
23     });
24
25     describe("listAddedSince", () => {
26         it("pages through results, returning collected ids and the latest historyId", () => {
27             const pages = [
28                 {
29                     history: [{ messagesAdded: [{ message: { id: "m1" } }] }],
30                     historyId: "100",
31                     nextPageToken: "p2",
32                 },
33                 {
34                     history: [{ messagesAdded: [{ message: { id: "m2" } }] }],
35                     historyId: "200",
36                 },
37             ];
38             let call = 0;
39             const list = jest.fn(() => pages[call++]);
40             g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
41
42             expect(listAddedSince("50")).toEqual({
43                 messageIds: ["m1", "m2"],
44                 historyId: "200",
45             });
46             expect(list).toHaveBeenCalledTimes(2);
47         });
48
49         it("returns null when the start historyId has expired", () => {
50             const list = jest.fn(() => {
51                 throw new Error("call failed with error: 404 not found");
52             });
53         });
54     });
55

```

```

52     });
53     g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
54     expect(listAddedSince("1")).toBeNull();
55   });
56
57   it("rethrows errors that are not history expiry", () => {
58     const list = jest.fn(() => {
59       throw new Error("backend error 500");
60     });
61     g.Gmail = { Users: { getProfile: jest.fn(), History: { list } } };
62     expect(() => listAddedSince("1")).toThrow("500");
63   });
64 });
65
66 describe("threadsForMessageIds", () => {
67   it("resolves distinct threads and skips messages that no longer exist", () => {
68     const threadA = { getId: () => "tA" };
69     const byId: Record<string, { getThread: () => { getId: () => string } }> = {
70       a: { getThread: () => threadA },
71       b: { getThread: () => threadA }, // same thread as a
72       c: { getThread: () => ({ getId: () => "tC" }) },
73     };
74     g.GmailApp = { getMessageById: (id: string) => byId[id] ?? null };
75
76     const threads = threadsForMessageIds(["a", "b", "c", "gone"]);
77     expect(threads.map((t) => (t as { getId: () => string }).getId()).sort()).toEqual(
78       ["tA", "tC"],
79     );
80   });
81 });
82

```

7. user (2026-06-10T16:26:34.987Z)

```

1    /*
2    * Pure helpers for the Gmail `users.history.list` incremental sync. These
3    * operate on plain data shapes (the JSON the Gmail advanced service returns)
4    * and have no GAS dependencies, so they are exercised directly in unit tests.
5    * The actual advanced-service calls live in the `gmailHistory` shell.
6    *
7    * See https://developers.google.com/workspace/gmail/api/guides/sync ? a stored
8    * `startHistoryId` returns only the changes since that point (~2 quota units vs
9    * ~5 for a list call). The id can expire, in which case the API responds 404
10   * and the caller must fall back to a full (date-window) sync.
11   */
12
13   /** A message stub as it appears inside a history record. */
14   export interface HistoryMessage {
15     id?: string;
16     threadId?: string;
17   }
18
19   /** One change record; we only care about messages added to the mailbox. */
20   export interface HistoryRecord {
21     messagesAdded?: { message?: HistoryMessage }[];
22   }
23
24   /** The shape of a `Gmail.Users.History.list` response we depend on. */
25   export interface HistoryListResponse {
26     history?: HistoryRecord[];
27     historyId?: string;
28     nextPageToken?: string;
29   }
30
31   /**

```

```

32     * Collects the unique ids of messages added across the given history pages, in
33     * first-seen order. Records without a `messagesAdded` entry (e.g. label-only
34     * changes) contribute nothing.
35     */
36     export function collectAddedMessageIds(
37         responses: HistoryListResponse[],
38     ): string[] {
39         const seen = new Set<string>();
40         const ids: string[] = [];
41         for (const response of responses) {
42             for (const record of response.history ?? []) {
43                 for (const added of record.messagesAdded ?? []) {
44                     const id = added.message?.id;
45                     if (id && !seen.has(id)) {
46                         seen.add(id);
47                         ids.push(id);
48                     }
49                 }
50             }
51         }
52         return ids;
53     }
54
55     /**
56     * The newest `historyId` to persist for the next run: the last page's id, or
57     * the supplied fallback (the start id) if no page reported one.
58     */
59     export function nextHistoryId(
60         responses: HistoryListResponse[],
61         fallback: string,
62     ): string {
63         for (let i = responses.length - 1; i >= 0; i--) {
64             const id = responses[i].historyId;
65             if (id) {
66                 return id;
67             }
68         }
69         return fallback;
70     }
71
72     /**
73     * Whether an error thrown by `history.list` means the stored `startHistoryId`
74     * has expired. Gmail responds with HTTP 404 ("Requested entity was not found")
75     * once the id predates the mailbox's retained history; the caller must then do
76     * a full sync. We match defensively on both the status code and the message
77     * text since the advanced service surfaces errors as a stringified exception.
78     */
79     export function isHistoryExpired(error: unknown): boolean {
80         const e = error as { message?: string; details?: { code?: number } } | null;
81         if (e && e.details && e.details.code === 404) {
82             return true;
83         }
84         const text = String(e?.message ?? error ?? "").toLowerCase();
85         return text.includes("404") || text.includes("not found");
86     }
87

```

8. user (2026-06-10T16:26:35.457Z)

```

1     /*
2     * The MIT License (MIT)
3     * Copyright (c) 2013 Avi Dullu
4     */
5
6     /** Top-level Drive folder that holds every synced attachment. */

```

```

7     export const GMAIL_ATTACHMENT_FOLDER = "GmailAttachments";
8
9     /** Public project / support page shown in the setup UI. */
10    export const SUPPORT_URL =
11        "https://github.com/avidullu/gmail-attachments-to-gdrive";
12
13    /** Privacy policy shown in the setup UI (and the OAuth consent screen). */
14    export const PRIVACY_POLICY_URL =
15        "https://github.com/avidullu/gmail-attachments-to-gdrive/blob/master/PRIVACY.md";
16
17    /**
18     * Apps Script enforces a daily quota on file creation. We cap ourselves below
19     * it so a single run can never exhaust the account's allowance.
20     */
21    export const MAX_FILES_ADD_PER_DAY = 240;
22
23    // Time constants, in milliseconds.
24    export const ONE_MINUTE = 60_000;
25    export const ONE_DAY = 86_400_000;
26    export const THIRTY_DAYS = 2_592_000_000;
27    export const ONE_YEAR = 31_622_400_000;
28    export const SIX_HOURS = 21_600_000;
29
30    /**
31     * Wall-clock budget for a single sync execution. Apps Script kills consumer
32     * scripts at 6 minutes, so we stop at 5 to leave margin for the final
33     * bookkeeping and trigger scheduling.
34     */
35    export const MAX_RUNTIME_MS = 5 * ONE_MINUTE;
36
37    /** Defensive upper bound on how many sync triggers may exist at once (limit is 20). */
38    export const MAX_SYNC_TRIGGERS = 12;
39
40    // Keys used in PropertiesService.getUserProperties().
41    export const PROP_FOLDER = "f";
42    export const PROP_LAST_SYNCED = "l";
43    export const PROP_FILES_ADDED_TODAY = "fa";
44    export const PROP_DEBUG = "dbg";
45    export const PROP_LAST_ERROR = "err";
46    export const PROP_PROCESSED_IDS = "pid";
47    export const PROP_RUN_STATS = "stats";
48    export const PROP_SUMMARY_LAST_SENT = "sumsent";
49    export const PROP_HISTORY_ID = "hid";
50
51    /** Flip to true to emit verbose Logger output during development. */
52    export const ENABLE_DEBUGGING = false;
53
54    /**
55     * Hybrid incremental sync. OFF by default so the proven date-window backfill
56     * remains the sole code path; flip to true to roll out the cheaper Gmail
57     * `history.list` steady state. When on, once a backfill catches up the current
58     * mailbox `historyId` is captured and subsequent runs process only newly-added
59     * messages, falling back to a date-window re-scan if the id expires (HTTP 404).
60     * Requires the Gmail advanced service (declared in appsscript.json); no extra
61     * OAuth scope ? `gmail.readonly` already covers history reads.
62     */
63    export const ENABLE_INCREMENTAL_SYNC = false;
64
65    /**
66     * Daily summary email. OFF by default ? enabling it requires the
67     * `script.send_mail` OAuth scope (a re-consent on next deploy). When enabled,
68     * set SUMMARY_EMAIL_RECIPIENT to the destination address (empty = the
69     * effective user's own address).
70     */
71    export const ENABLE_SUMMARY_EMAIL = false;

```

```
72     export const SUMMARY_EMAIL_RECIPIENT = "";
73
```

9. user (2026-06-10T16:27:21.032Z)

```
a764030 Codify the auditability invariant (Option A) + enforce no egress (#14)
90c85ba Merge pull request #13 from avidullu/docs/vision-and-architecture
4975254 docs: capture extraction-pivot vision and architecture
172b3b3 Validate setup input + traceable attachment-copy errors (#12)
964b58c Portable attachment names + robust content-type matching (#11)
5e934b1 Make the web app multi-user-ready for small-group / org deployment (#10)
e4362a3 Add hybrid history.list incremental sync (opt-in) (#9)
6d0b10e ci: trigger deploy to verify CLASPRC_JSON (v2 format)
src/config.ts:63:export const ENABLE_INCREMENTAL_SYNC = false;
src/sync.ts:2:  ENABLE_INCREMENTAL_SYNC,
src/sync.ts:160:    // Incremental mode is opt-in (ENABLE_INCREMENTAL_SYNC) so it rolls out
src/sync.ts:163:    const historyId = ENABLE_INCREMENTAL_SYNC ? state.getHistoryId() : null;
src/sync.ts:195:    const candidateHistoryId = ENABLE_INCREMENTAL_SYNC
tests/syncIncremental.test.ts:4:  ENABLE_INCREMENTAL_SYNC: true,
src/gmailHistory.ts:71:    const message = GmailApp.getMessageById(id);
tests/gmailHistory.test.ts:74:    g.GmailApp = { getMessageById: (id: string) => byId[id] ??
null };
tests/setup.ts:23:g.GmailApp = { search: jest.fn(() => []), getMessageById: jest.fn() };
tests/syncIncremental.test.ts:65:  g.GmailApp = { search: jest.fn(() => []), getMessageById:
jest.fn() };
tests/syncIncremental.test.ts:105:    (g.GmailApp as { getMessageById: jest.Mock
}).getMessageById = jest.fn()
```

10. user (2026-06-10T16:27:27.251Z)

```
1      // Enable the opt-in incremental path for this suite only.
2      jest.mock("../src/config", () => ({
3        ...jest.requireActual("../src/config"),
4        ENABLE_INCREMENTAL_SYNC: true,
5      }));
6
7      import {
8        ONE_DAY,
9        PROP_DEBUG,
10       PROP_FILES_ADDED_TODAY,
11       PROP_FOLDER,
12       PROP_HISTORY_ID,
13       PROP_LAST_SYNCED,
14       PROP_PROCESSED_IDS,
15     } from "../src/config";
16     import { syncUserDrives } from "../src/sync";
17
18     type Store = Record<string, string>;
19     const g = globalThis as unknown as Record<string, unknown>;
20     const nowIso = () => new Date().toJSON();
21
22     function fakeProps(initial: Store) {
23       const store: Store = { ...initial };
24       return {
25         store,
26         getProperty: (k: string) => (k in store ? store[k] : null),
27         setProperty: (k: string, v: string) => {
28           store[k] = String(v);
29         },
30         deleteProperty: (k: string) => {
31           delete store[k];
32         },
33         deleteAllProperties: () => {
34           for (const k of Object.keys(store)) delete store[k];
35         },

```

```

36     };
37 }
38
39 function fakeScriptApp() {
40     const triggers: { getHandlerFunction: () => string }[] = [];
41     return {
42         newTrigger: (handler: string) => ({
43             timeBased: () => ({
44                 after: () => ({
45                     create: () => triggers.push({ getHandlerFunction: () => handler }),
46                 }),
47             }),
48         }),
49         getProjectTriggers: () => [...triggers],
50         deleteTrigger: () => undefined,
51     };
52 }
53
54 function iter<T>(items: T[]) {
55     let i = 0;
56     return { hasNext: () => i < items.length, next: () => items[i++] };
57 }
58
59 function installBase(props: ReturnType<typeof fakeProps>) {
60     g.PropertiesService = { getUserProperties: () => props };
61     g.ScriptApp = fakeScriptApp();
62     g.LockService = {
63         getUserLock: () => ({ tryLock: () => true, releaseLock: jest.fn() }),
64     };
65     g.GmailApp = { search: jest.fn(() => []), getMessageById: jest.fn() };
66 }
67
68 function baseProps(overrides: Store = {}) {
69     return fakeProps({
70         [PROP_DEBUG]: "1",
71         [PROP_FOLDER]: "byContentType",
72         [PROP_LAST_SYNCED]: nowIso(),
73         [PROP_FILES_ADDED_TODAY]: JSON.stringify({ added: 0, date: nowIso() }),
74         ...overrides,
75     });
76 }
77
78 describe("syncUserDrives ? incremental mode", () => {
79     it("processes messages added since the stored historyId and advances it", () => {
80         const props = baseProps({ [PROP_HISTORY_ID]: "100" });
81         installBase(props);
82
83         // A Drive head folder that accepts one file.
84         const head = {
85             getName: () => "GmailAttachments",
86             getFolders: () => iter([]),
87             createFolder: () => head,
88             getFilesByName: () => iter([]),
89             createFile: () => ({ setDescription: () => undefined }),
90         };
91         g.DriveApp = { getFolders: () => iter([]), createFolder: () => head };
92
93         const message = {
94             getId: () => "m1",
95             getFrom: () => "alice@example.com",
96             getDate: () => new Date(2012, 5, 6),
97             getAttachments: () => [
98                 {
99                     getContentType: () => "application/pdf",
100                     getName: () => "report.pdf",

```

```

101         copyBlob: () => ({ setName: () => undefined }),
102     },
103 ],
104 };
105 (g.GmailApp as { getMessageById: jest.Mock }).getMessageById = jest.fn(
106     () => ({ getThread: () => ({ getId: () => "t1", getMessages: () => [message] })
107 })),
108 );
109 g.Gmail = {
110     Users: {
111         getProfile: jest.fn(),
112         History: {
113             list: jest.fn(() => ({
114                 history: [{ messagesAdded: [{ message: { id: "m1" } }] }]),
115                 historyId: "200",
116             })),
117         },
118     },
119 };
120 syncUserDrives();
121
122 // Never falls back to a date-window search while the history is valid.
123 expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
124 expect(props.store[PROP_HISTORY_ID]).toBe("200");
125 expect(JSON.parse(props.store[PROP_PROCESSED_IDS])).toContain("m1");
126 });
127
128 it("clears the cursor and rewinds lastSynced when the history expired", () => {
129     const props = baseProps({ [PROP_HISTORY_ID]: "1" });
130     installBase(props);
131     g.Gmail = {
132         Users: {
133             getProfile: jest.fn(),
134             History: {
135                 list: jest.fn(() => {
136                     throw new Error("call failed with error: 404 not found");
137                 }),
138             },
139         },
140     };
141     syncUserDrives();
142
143     expect(PROP_HISTORY_ID in props.store).toBe(false);
144     // lastSynced rewound to roughly 30 days ago for the recovery re-scan.
145     const lastSynced = Date.parse(props.store[PROP_LAST_SYNCED]);
146     expect(Date.now() - lastSynced).toBeGreaterThan(29 * ONE_DAY);
147     // No history ? no incremental processing this run; backfill resumes next run.
148     expect((g.GmailApp as { search: jest.Mock }).search).not.toHaveBeenCalled();
149 });
150
151
152 it("captures the mailbox historyId once backfill catches up", () => {
153     // No stored historyId ? backfill mode; lastSynced=now means we are already
154     // caught up, so the run captures the current historyId for next time.
155     const props = baseProps();
156     installBase(props);
157     g.Gmail = {
158         Users: {
159             getProfile: jest.fn(() => ({ historyId: "555" })),
160             History: { list: jest.fn() },
161         },
162     };
163     syncUserDrives();

```

```
165
166     expect((g.GmailApp as { search: jest.Mock }).search).toHaveBeenCalled();
167     expect(props.store[PROP_HISTORY_ID]).toBe("555");
168   });
169 });
170
```

11. user (2026-06-10T16:28:07.213Z)

Structured output provided successfully

12. assistant (2026-06-10T16:28:12.295Z)

Verification complete. The structured output has been submitted: the bug is real (the code assumes a null return where the real GAS runtime throws), but the claimed "high" severity is refuted by existing mitigations ? the incremental path is behind an off-by-default feature flag, and the system self-heals via history-id expiry plus a 30-day rewind. Adjusted to medium.